

Deep Learning para Business Analytics

Día 03 · Tema 1 — Buenas prácticas

Cómo entrenar redes sin que se mientan a sí mismas.

Eduardo C. Garrido-Merchán

Profesor Propio Adjunto, Métodos Cuantitativos, ICADE

ecgarrido@comillas.edu



Madrid, Día 03

BBVA  NETFLIX amazon INDITEX 

ejemplos del día

Lo que vamos a cubrir hoy.

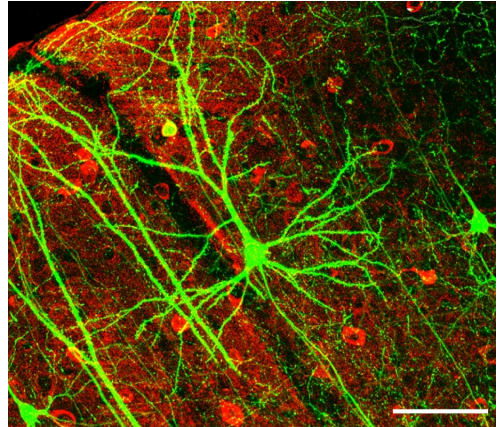
1. Regularización
2. Learning rate schedules
3. Dashboards y trackers de experimentos
4. Caso completo: receta exprés

Lo que vas a saber hacer al final:

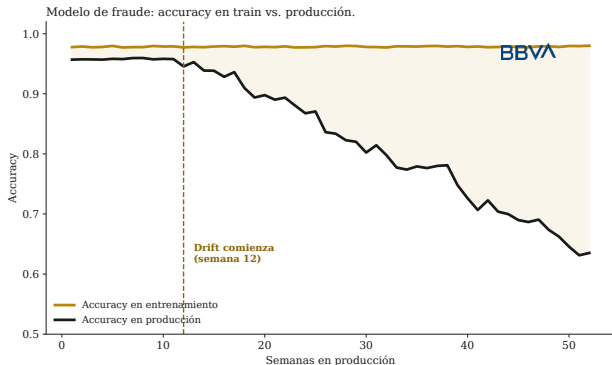
1. Diagnosticar overfitting y aplicar dropout, BN y weight decay.
2. Programar un cosine annealing del LR en PyTorch en cinco líneas.
3. Defender la elección de tracker (W&B vs MLflow) ante un equipo de datos.

Parte 1 / 4

Regularización



El modelo de fraude de BBVA dejó de funcionar a los tres meses.



Qué pasó. ⚠ El modelo memorizó patrones que eran artefactos de la ventana de entrenamiento: un pico estacional de compras navideñas, un partner temporal, una campaña puntual.

Resultado. 📊 Accuracy en train: 98 %.
Accuracy en producción a las 12 semanas: 72 % y cayendo.

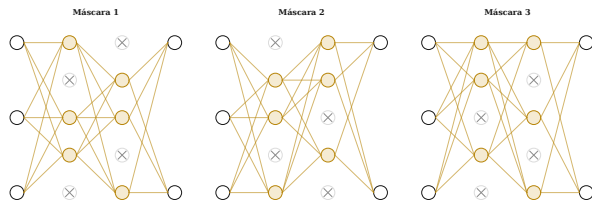
BBVA Caso BBVA (estilizado)

Lección. 🛡 Regularizar = comprar seguros contra la memorización. Un modelo que generaliza es un modelo que sobrevive en producción.

IDEA CLAVE. El overfitting no es solo un problema académico: cuesta dinero real.

Dropout es estudiar para el examen tapándose los apuntes al azar.

Dropout: cada batch entrena una sub-red distinta.



Analogía. 🎲 Si aprendes a resolver problemas sin depender de ningún compañero concreto, eres más robusto ante ausencias. Dropout hace exactamente eso con las neuronas.

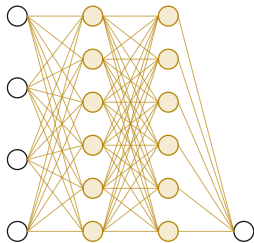
Mecanismo. ⚙️ En cada minibatch, cada neurona oculta se apaga con probabilidad p . La red que queda es una sub-red distinta. El entrenamiento promedia implícitamente sobre 2^H sub-redes (H = neuronas ocultas).

Efecto. 🛡️ Fuerza representaciones redundantes: ninguna neurona individual puede “cargar” con toda la señal. Reduce co-adaptación.

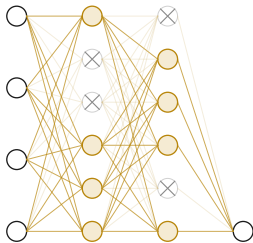
IDEA CLAVE. Dropout es el ensemble más barato que existe: 2^H modelos por el precio de uno.

Dropout: apagar neuronas al azar en cada batch.

Sin dropout (entrenamiento)



Con dropout $p=0.33$ (entrenamiento)

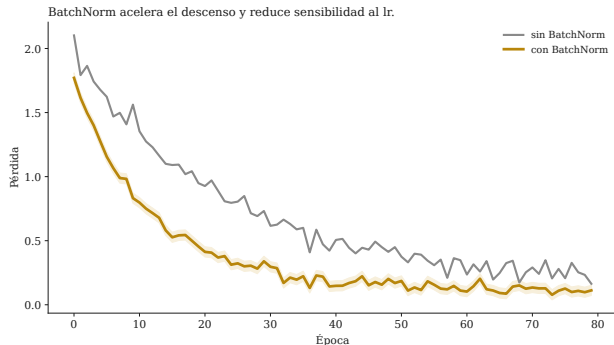


Idea. 🎯 Cada neurona se desactiva con probabilidad p (0,1 a 0,5) en entrenamiento. En inferencia se conservan todas.

Efecto. 🛡 La red no puede depender de neuronas específicas; aprende representaciones redundantes.

IDEA CLAVE. Dropout es el regularizador más simple y más efectivo en MLP y CNN.

BatchNorm: estabilizar la distribución entre capas.



Idea. 📖 Normaliza las activaciones por minibatch (media 0, desv. 1) y aprende dos parámetros (γ , β) por canal.

Efecto. 🚀 Permite lr más alto, acelera el entrenamiento, suaviza la pérdida y reduce la sensibilidad a la inicialización.

Cuándo no. ⚠ Si el batch es muy pequeño (≤ 4), usa LayerNorm o GroupNorm.

IDEA CLAVE. Por defecto: BatchNorm en CNN, LayerNorm en Transformer.

BatchNorm paso a paso sobre un minibatch de 4 valores.

BatchNorm paso a paso: minibatch de 4 valores.

Activaciones crudas	Centradas ($-\mu$)	Normalizadas ($\div \sigma$)	Escaladas ($\gamma \cdot \hat{x} + \beta$)
2.00	-3.00	-1.34	-1.51
4.00	-1.00	-0.45	-0.17
6.00	1.00	0.45	1.17
8.00	3.00	1.34	2.51
$\mu_B = 5$	$\sigma_B^2 = 5$	$\sigma_B = 2.24$	$\gamma = 1.5, \beta = 0.5$

Datos. ⚙️

Minibatch $a = (2, 4, 6, 8)$.

Paso 1 — media y varianza. 🧠

$$\mu_B = \frac{2+4+6+8}{4} = 5$$

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5$$

Paso 2 — normalizar. 🎯

$$\hat{a}_i = \frac{a_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \approx \frac{a_i - 5}{2.236}$$

$$\hat{a} = (-1.34, -0.45, 0.45, 1.34)$$

Paso 3 — escalar y desplazar. 🚀

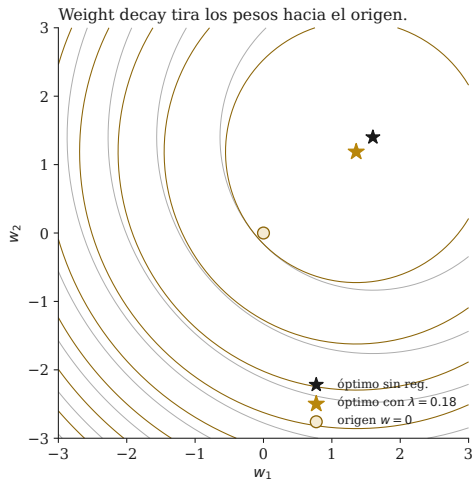
Con $\gamma = 1.5, \beta = 0.5$:

$$y_i = \gamma \hat{a}_i + \beta$$

$$y = (-1.51, -0.17, 1.17, 2.51)$$

IDEA CLAVE. γ y β se aprenden: la red puede “des-hacer” la normalización si lo necesita.

Weight decay: añadir un coste por pesos grandes.



La pérdida pasa a ser:

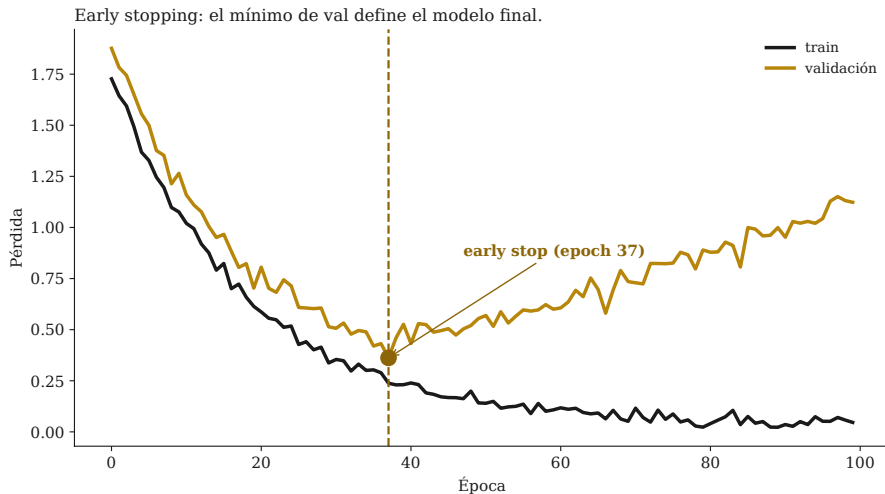
$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \|w\|_2^2$$

Por qué funciona. ⚖ Penaliza soluciones complejas; equivale a prior gaussiana sobre los pesos.

En PyTorch. ⚙ AdamW(params, weight_decay=1e-4).

IDEA CLAVE. Valor típico: $\lambda \in [10^{-5}, 10^{-3}]$.

Early stopping: parar antes de empezar a memorizar.



Ejemplo a mano: trazado paso a paso de early stopping.

Trazo de un early stopping con paciencia 12 sobre 28 épocas.

Época	Train loss	Val loss	best?	since_best	Acción
1	0.86	0.71	sí	0	guardar checkpoint
5	0.52	0.45	sí	0	guardar
10	0.34	0.34	sí	0	guardar
15	0.20	0.35	no	1	seguir entrenando
18	0.14	0.41	no	4	seguir
22	0.10	0.48	no	8	seguir, atención
28	0.07	0.62	no	14	¡STOP! (patience=12)

Concepto → aplicación: qué regularizador uso y para qué.

Técnica	Para qué problema	Entrada / Salida	Cuándo evitar
Dropout	MLP, CNN clásicas, modelos tabulares medianos.	Activaciones → activaciones con ruido.	Modelos muy pequeños, BN fuerte.
BatchNorm	CNN sobre imágenes, MLP densos.	Activaciones → activaciones normalizadas.	$\text{Batch} \leq 4$, modelos pre-entrenados.
Weight decay	Cualquier red, casi siempre.	Pesos → término extra en la pérdida.	Modelos sub-ajustados (λ alto).
Early stop	Cualquier red, casi siempre.	Curva val → checkpoint final.	Dataset trivial, sin curva clara.
LR cosine	Red moderna con AdamW.	Iteración → lr instantáneo.	Datasets muy pequeños.

IDEA CLAVE. Combinar los cinco es el primer prototipo correcto del Bloque I.

Parte 2 / 4

Learning rate schedules

Caffeine keeps you awake by blocking adenosine receptors.
Normally, when adenosine binds to its receptors,
neural activity slows down and you feel sleepy.

[C1=NC2=C(N1)C=NC2=O] [C1=NC2=C(N1)C=NC2=O]

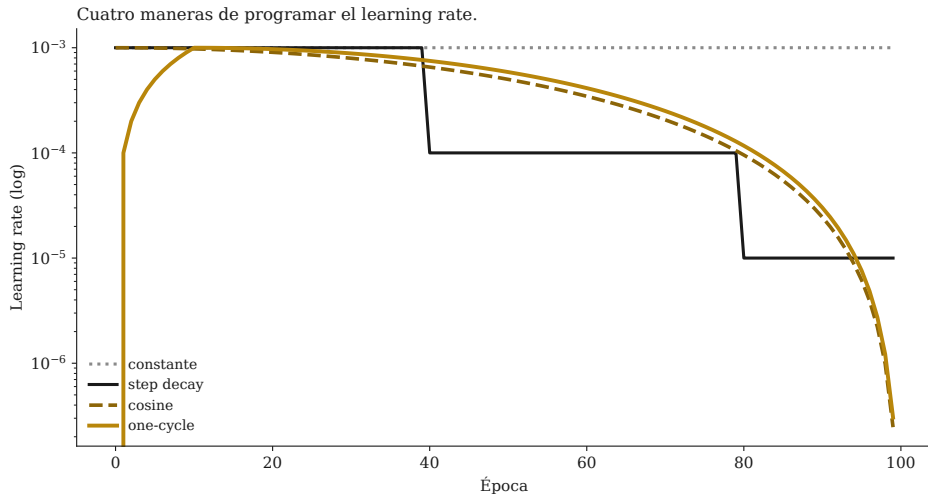
Caffeine Adenosine



When caffeine binds to those receptors,
it blocks their activity, allowing neural activity
to speed up, so you feel alert!

52 BRAIN FACTS
KNOWING NEURONS

Cuatro maneras de programar el learning rate.



Cuándo usar cada schedule.

Schedule	Idea	Cuándo
Constante	lr fijo durante todo el entrenamiento.	Prototipos, datasets pequeños, primer experimento.
Step decay	Divide lr por 10 cada N épocas.	Histórico (CNN sobre ImageNet con SGD).
Cosine	Decae suavemente como medio coseno.	Defecto moderno con AdamW.
One-cycle	Warmup + cosine: sube y luego baja.	Para acelerar entrenamiento en pocas épocas.

IDEA CLAVE. Si te bloqueas con el lr, empieza con cosine. Es el menos malo en casi todos los problemas.

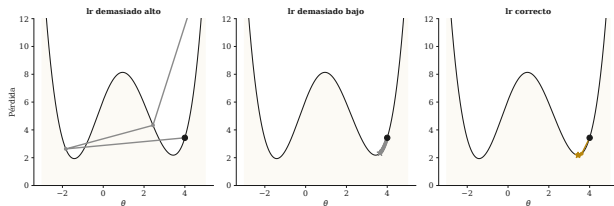
Programar cosine en PyTorch en cinco líneas.

```
from torch.optim.lr_scheduler import CosineAnnealingLR
opt = torch.optim.AdamW(m.parameters(), lr=1e-3,
weight_decay=1e-4)
sched = CosineAnnealingLR(opt, T_max=epochs)
for ep in range(epochs):
    for xb, yb in train_loader:
        opt.zero_grad()
        loss_fn(m(xb), yb).backward()
        opt.step()
    sched.step()
```

IDEA CLAVE. Esta línea sola sube la AUC de validación 2–5 % sin tocar el modelo.

El learning rate controla el balance exploración-explotación.

El learning rate controla la trayectoria de optimización.



lr alto = varianza alta. ⚠ Los parámetros saltan entre regiones de la superficie. Puede escapar de mínimos locales, pero también divergir.

lr bajo = sesgo alto. 🎯 Los parámetros apenas se mueven desde la inicialización. Riesgo de quedarse en un mínimo local cercano al punto de partida.

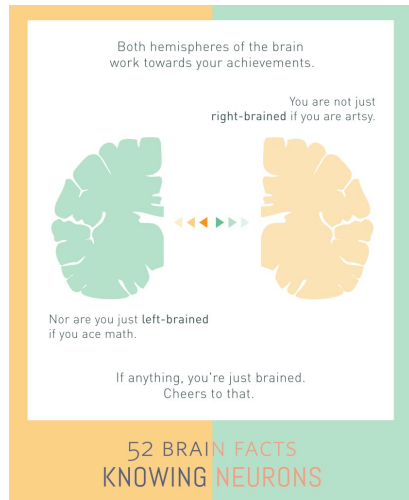
lr correcto. 🚀 Paso suficiente para explorar al inicio, cada vez más fino para explotar. Esto es lo que hace un **schedule cosine**.

Warmup. 🔥 Al inicio, las estadísticas de BatchNorm no han estabilizado. Subir el lr gradualmente evita explosiones tempranas.

IDEA CLAVE. Exploración temprana (lr alto) y explotación tardía (lr bajo).

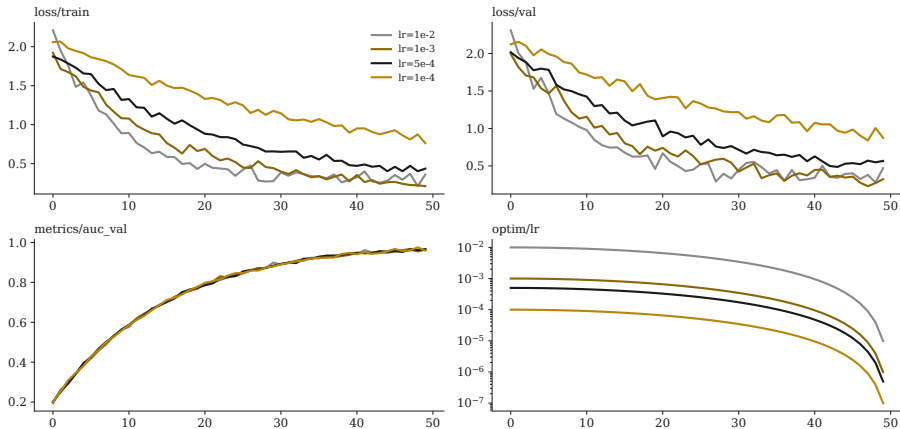
Parte 3 / 4

Dashboards y trackers de experimentos



Cuatro paneles que debes ver siempre.

Tablero de experimentos: 4 corridas, 4 paneles.



Qué herramienta para qué aplicación.

Herramienta	Para qué	Coste	Cuándo
TensorBoard	Logging local básico (loss, scalars, imágenes).	Gratis	Sesiones individuales, casi cualquier proyecto PyTorch.
Weights & Biases	Comparar experimentos en cloud con dashboard rico.	Gratis < 100GB	Trabajo en equipo, búsquedas de hiperparámetros.
MLflow	Tracking + model registry + serving open-source.	Gratis (self-host)	Producción empresarial con varios usuarios.
Comet	Alternativa cloud a W&B con énfasis en NLP.	Gratis < 100GB	Equipos NLP / Hugging Face.

IDEA CLAVE. En el proyecto AI-first todo equipo elige uno. Sin tracking no hay reproducibilidad ni defensa oral.

Antipatrón: el random search de las 03:14.

La trampa. ⚠ Lanzar 50 configuraciones a saco y elegir la mejor. Suele costar muchas horas y entregar el mismo resultado que un solo run bien planteado.

La regla. 🎯 Antes de barrer, fija *modelo, pérdida, optimizador y schedule*. Luego mueve un único hiperparámetro a la vez.

Cuándo sí. 🔍 Búsquedas estructuradas (Bayesian optimization, hyperband) sobre 3–4 hiperparámetros con presupuesto cerrado.

IDEA CLAVE. Cada hora invertida en un dashboard ahorra una semana de búsqueda ciega.

Parte 4 / 4

Caso completo: receta exprés

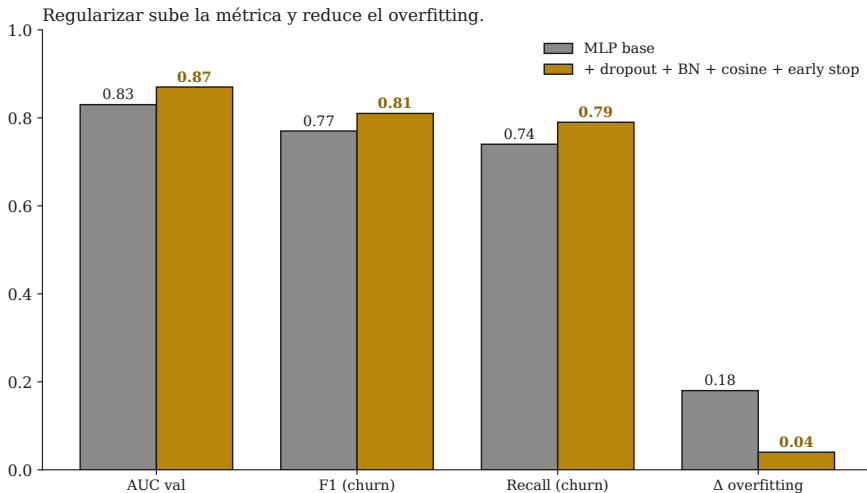


Receta exprés del "primer prototipo correcto".

1. **Arquitectura.** 🧠 2–3 capas ocultas, ancho 32–128, ReLU. Empieza pequeño.
2. **Optimizador.** 🚀 **AdamW** con $lr = 10^{-3}$ y $weight_decay = 10^{-4}$.
3. **Schedule.** 🕒 **CosineAnnealingLR** con $T_{max} = epochs$.
4. **Regularización.** 🛡️ **Dropout** $p \in \{0, 1, 0.3\}$, **BatchNorm** tras cada Linear (excepto última capa).
5. **Stopping.** ✅ **Early stopping** con $patience=10$ sobre val.
6. **Tracking.** 📊 W&B (o MLflow). Loguea `loss/train`, `loss/val`, `metric/auc`, `optim/lr`.
7. **Validación.** 🎯 5-fold antes de declarar nada.

IDEA CLAVE. Esta receta es el "primer prototipo correcto" del Bloque I. Aprended bien hoy y copiad cuatro veces en el semestre.

Mismo dataset, mismo modelo, regularización añadida.



Lo que el alumno se lleva a casa de este día.

Concepto 1. 🎯 Dropout y 📖 BatchNorm son los regularizadores por defecto.

Concepto 2. 🕒 Cosine annealing del lr mejora casi todo entrenamiento sin coste adicional.

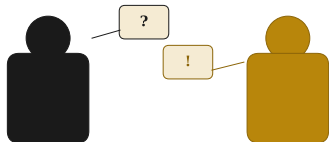
Concepto 3. ✅ Early stopping cuesta cero y previene la mitad del overfitting.

Concepto 4. 👁 Sin dashboard no hay reproducibilidad ni decisiones defendibles.

Para la práctica. Tomar el MLP del Día 02, aplicar la receta y comparar.

Próximo día. Día 04 (Tema 2): CNN para visión y retail.

Pausa para debatir: ¿Mata la receta exprés la creatividad técnica?



La pregunta. 💡 Cosine + Dropout + BatchNorm + EarlyStopping es "el primer prototipo correcto". ¿Es educativo o cierra la exploración?

Posición A. ⚠️ Mata: sin probar SGD puro no se entiende por qué Adam funciona.

Posición B. ✅ No mata: la receta es el punto de partida, no la meta. Lo que pierde es tiempo no aprendizaje.

Reglas. 5 minutos, dos turnos de palabra por bando, móviles boca abajo. Yo modero, no participo.

Cierre del Tema 1.

*Próxima clase: convolución, pooling,
transfer learning.*

Eduardo C. Garrido-Merchán

ecgarrido@comillas.edu

